GUIA TÉCNICA PARA PROVA

Telas • Servidor • Banco de Dados • API • CRUD

Sequência para não se perder: **Banco** → **Servidor/API** → **Telas** → **Integração** → **Testes**.

> **Regra de ouro:** faça uma parte funcionar antes da próxima.

1. VISÃO GERAL

Camada	Função	Exemplos
Front-end	Interface	HTML, CSS, JavaScript
Back-end	Regras, validações e API	PHP, Node.js, Python
Banco	Armazena dados	MySQL, PostgreSQL, MongoDB

> **Fluxo:** Usuário → Tela → HTTP → API → Banco → resposta → Tela.

2. TELAS

• Login: usuário/e-mail + senha + entrar.

• Listagem: tabela/cards + Editar + Excluir.

• Cadastro: formulário com campos obrigatórios.

• Edição: formulário preenchido.

• Detalhes: informações completas, se necessário.

> **CRUD** = Create, Read, Update, Delete.

3. HTML ESSENCIAL

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header>...</header>
  <main>...</main>
  <script src="script.js"></script>
</body>
</html>
```

Formulário:

```
<form id="formCurso">
  <label>Nome</label>
  <input type="text" name="nome" required>
  <label>Descrição</label>
  <textarea name="descricao"></textarea>
  <button type="submit">Salvar</button>
</form>
```

4. CSS

```
* { box-sizing: border-box; margin: 0; padding: 0; }
body { font-family: Arial; background: #f5f5f5; }
.container { max-width: 1200px; margin: auto; padding: 24px; }
.grid { display: grid; grid-template-columns: repeat(3,1fr); gap: 20px; }
.card { background: white; padding: 20px; border-radius: 12px; }
```

> **Decore:** flex = linha/coluna; grid = grade; gap = espaço entre itens; padding = interno; margin = externo.

5. BANCO DE DADOS

Exemplo: sistema de cursos.

• curso: id, nome, descrição, categoria, carga_horaria, turno, modalidade, valor, data_inicio, status.

• usuario: id, nome, email, senha, tipo.

• categoria: id, nome.

> **Tipos:** INT = número; VARCHAR = texto curto; TEXT = texto grande; DATE = data; DECIMAL = dinheiro; BOOLEAN = sim/não.

6. SQL PRINCIPAL

```
CREATE TABLE curso (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nome VARCHAR(150) NOT NULL,
  descricao TEXT,
  carga_horaria INT,
  valor DECIMAL(10,2),
  data_inicio DATE,
  status VARCHAR(30)
);

INSERT INTO curso (nome, carga_horaria)
VALUES ('Informática', 160);

SELECT * FROM curso;
SELECT * FROM curso WHERE id = 1;

UPDATE curso
SET nome = 'Informática Básica'
WHERE id = 1;

DELETE FROM curso
WHERE id = 1;
```

7. CHAVES E RELACIONAMENTOS

• PK (Primary Key): identifica um registro. Ex.: curso.id.

• FK (Foreign Key): aponta para a PK de outra tabela.

• 1:N: uma categoria possui vários cursos.

• N:N: vários A com vários B; geralmente usa tabela intermediária.

> Exemplo: **categoria.id** → **curso.categoria_id**.

8. SERVIDOR / API

O servidor recebe requisições, valida dados, aplica regras, acessa o banco e devolve resposta.

Método	Rota	Ação
GET	/cursos	Listar
GET	/cursos/1	Buscar um
POST	/cursos	Criar
PUT/PATCH	/cursos/1	Editar
DELETE	/cursos/1	Excluir

9. STATUS HTTP

• 200 OK = sucesso.

• 201 Created = criado.

• 400 Bad Request = dados inválidos.

• 401 Unauthorized = não autenticado.

• 403 Forbidden = sem permissão.

• 404 Not Found = não encontrado.

• 500 Internal Server Error = erro no servidor.

10. JSON / API

```
POST /cursos
Content-Type: application/json

{
  "nome": "Informática Básica",
  "carga_horaria": 160,
  "valor": 0
}

Resposta:
{
  "id": 1,
  "nome": "Informática Básica"
}
```

11. JAVASCRIPT + FETCH

```
const form = document.querySelector("#formCurso");

form.addEventListener("submit", async (e) => {
  e.preventDefault();

  const dados = {
    nome: document.querySelector("[name=nome]").value
  };

  const resposta = await fetch("http://localhost:3000/cursos", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(dados)
  });
});
```

12. ESTRUTURA DE PASTAS

```
projeto/
■■■ frontend/
■   ■■■ index.html
■   ■■■ cadastro.html
■   ■■■ style.css
■   ■■■ script.js
■■■ backend/
■   ■■■ routes/
■   ■■■ controllers/
■   ■■■ models/
■   ■■■ services/
■   ■■■ server.js
■■■ database/
    ■■■ script.sql
```

Ideia: rota recebe → controller organiza → service aplica regra → model/banco persiste.

13. CHECKLIST DA PROVA

■ Ler o enunciado e marcar entidades, campos e ações.

■ Listar as telas.

■ Definir tabelas, PKs e FKs.

■ Criar banco e testar INSERT + SELECT.

■ Criar servidor e testar uma rota.

■ Implementar CRUD.

■ Criar HTML/CSS.

■ Conectar formulário com API.

■ Adicionar validações.

■ Testar cadastrar, listar, editar e excluir.

■ Revisar IDs, campos e endpoints.

14. SE DER BRANCO

1 — Dados: o que precisa ser salvo? Defina entidades, campos, IDs e relacionamentos.

2 — Ações: o usuário cria, consulta, edita ou exclui? Pense no endpoint e SQL.

3 — Tela: formulário para entrada e tabela/cards para saída.

4 — Integração: HTML → JS/fetch → API → banco → JSON → HTML.

5 — Teste: faça um cadastro e confira banco + listagem.

15. MINI PROJETO — CURSOS

• Tela: cadastro + listagem.

• Campos: nome, categoria, carga horária, descrição, turno, valor, modalidade e data.

• Banco: curso + categoria.

• API: GET/POST/PUT/DELETE /cursos.

• Cadastro: formulário → POST → validação → INSERT → 201.

• Listagem: GET → SELECT → cards/tabela.

• Edição: PUT → UPDATE.

• Exclusão: DELETE → DELETE.

16. PEGADINHAS

• Nunca esquecer WHERE no UPDATE/DELETE.

• FK deve apontar para PK/UNIQUE.

• Não misturar nomes de campos sem planejamento.

• Validar no servidor, não só no HTML.

• Ao enviar JSON, usar Content-Type: application/json.

• Testar cada etapa isoladamente.

17. MAPA MENTAL

TELA → formulário/listagem → **JS** → fetch → **API** → rota/controller → **REGRA** → validação → **BANCO** → SQL → **JSON** → tela.

18. FRASES PARA DECORAR

CRUD: Criar, Ler, Atualizar, Deletar.

GET pega. **POST** cria. **PUT/PATCH** altera. **DELETE** remove.

PK identifica. **FK** relaciona. **Front** mostra. **Back** decide. **Banco** guarda.

RESUMO FINAL

ENTIDADES → TABELAS → CRUD → ENDPOINTS → TELAS → FETCH → TESTES.

Se você conseguir seguir essa ordem, consegue estruturar a maior parte de uma prova prática de desenvolvimento de sistemas.